

Module 3 - Database Integration - PostgreSQL, MySQL & MongoDB

Given data - Common Scenario: Student Course Registration System

HANDS-ON 3 - Advanced SQL — Subqueries, Views & Transaction

35. Students enrolled in more courses than the average enrollments per student

SQL

```
SELECT s.student_id,  
       s.first_name,  
       s.last_name  
FROM students s  
JOIN enrollments e  
ON s.student_id = e.student_id  
GROUP BY s.student_id  
HAVING COUNT(e.course_id) >  
(  
    SELECT AVG(course_count)
```

```
FROM

(
    SELECT COUNT(*) AS course_count
    FROM enrollments
    GROUP BY student_id
) avg_table
);
```

EXAMPLE TABLE

Input

Run SQL

```
SELECT s.student_id,
       s.first_name,
       s.last_name
FROM students s
JOIN enrollments e
ON s.student_id = e.student_id
GROUP BY s.student_id
HAVING COUNT(e.course_id) >
(
    SELECT AVG(course_count)
    FROM
    (
        SELECT COUNT(*) AS course_count
        FROM enrollments
        GROUP BY student_id
    ) avg_table
)
```

Output

student_id	first_name	last_name
1	Arjun	Mehta
2	Priya	Suresh
5	Vikram	Das
8	Deepika	Rao

36. Courses where all enrolled students received grade 'A'

SQL

```
SELECT c.course_name
```

```
FROM courses c
```

```
WHERE NOT EXISTS
```

```
(
```

```
    SELECT *
```

```
    FROM enrollments e
```

```
    WHERE e.course_id = c.course_id
```

```
        AND e.grade <> 'A'
```

```
);
```

EXAMPLE TABLE

The screenshot shows a SQL query editor with the following query:

```
SELECT c.course_name
FROM courses c
WHERE NOT EXISTS
(
  SELECT *
  FROM enrollments e
  WHERE e.course_id = c.course_id
  AND e.grade <> 'A'
);
```

The output section shows a table with one row:

course_name
Thermodynamics

37. Professor with the highest salary in each department

SQL

SELECT p.prof_name,

p.salary,

p.department_id

FROM professors p

WHERE p.salary =

(

SELECT MAX(p2.salary)

FROM professors p2

WHERE p2.department_id = p.department_id

);

EXAMPLE TABLE

Input

```
SELECT p.prof_name,
       p.salary,
       p.department_id
FROM professors p
WHERE p.salary =
(
  SELECT MAX(p2.salary)
  FROM professors p2
  WHERE p2.department_id = p.department_id
);
```

Run SQL

Output

prof_name	salary	department_id
Dr. Anand Krishnan	95000	1
Dr. Sunil Rajan	82000	2
Dr. Latha Gopal	79000	3
Dr. Kartik Bose	76000	4

38. Departments whose average professor salary exceeds 85000

SQL

SELECT dept_name,

avg_salary

FROM

(

SELECT d.dept_name,

```
    AVG(p.salary) AS avg_salary
FROM departments d
JOIN professors p
ON d.department_id = p.department_id
GROUP BY d.department_id, d.dept_name
) dept_avg
WHERE avg_salary > 85000;
```

EXAMPLE TABLE

Input

```
SELECT dept_name,
        avg_salary
FROM
(
    SELECT d.dept_name,
           AVG(p.salary) AS avg_salary
    FROM departments d
    JOIN professors p
    ON d.department_id = p.department_id
    GROUP BY d.department_id, d.dept_name
) dept_avg
WHERE avg_salary > 85000;
```

Output

dept_name	avg_salary
Computer Science	91500

Expected Outcome: Query 1 should return students enrolled in 3 or more courses based on sample data.

Task 2: Creating and Using Views Goal: Create reusable views that simplify complex queries

39. Create vw_student_enrollment_summary

SQL

CREATE VIEW vw_student_enrollment_summary AS

SELECT

s.student_id,

s.first_name || ' ' || s.last_name AS student_name,

d.dept_name,

COUNT(e.course_id) AS total_courses,

ROUND(

AVG(

CASE

WHEN e.grade='A' THEN 4

WHEN e.grade='B' THEN 3

WHEN e.grade='C' THEN 2

WHEN e.grade='D' THEN 1

WHEN e.grade='F' THEN 0

```
        END
    ),2
) AS GPA
FROM students s
JOIN departments d
ON s.department_id = d.department_id
LEFT JOIN enrollments e
ON s.student_id = e.student_id
GROUP BY s.student_id;
```

EXAMPLE TABLE

Input

```
CREATE VIEW vw_student_enrollment_summary AS
SELECT
    s.student_id,
    s.first_name || ' ' || s.last_name AS student_name,
    d.dept_name,
    COUNT(e.course_id) AS total_courses,
    ROUND(
        AVG(
            CASE
                WHEN e.grade='A' THEN 4
                WHEN e.grade='B' THEN 3
                WHEN e.grade='C' THEN 2
                WHEN e.grade='D' THEN 1
                WHEN e.grade='F' THEN 0
            END
        ),2
    )
```

Run SQL

Output

SQL query successfully executed. However, the result set is empty.

40. Create vw_course_stats

CREATE VIEW vw_course_stats AS

SELECT

c.course_name,

c.course_code,

COUNT(e.student_id) AS total_enrollments,

ROUND(

AVG(

CASE

WHEN e.grade='A' THEN 4

WHEN e.grade='B' THEN 3

WHEN e.grade='C' THEN 2

WHEN e.grade='D' THEN 1

WHEN e.grade='F' THEN 0

END

),2

) AS avg_gpa

FROM courses c

LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id;
EXAMPLE TABLE

Input

```
CREATE VIEW vw_course_stats AS
SELECT
  c.course_name,
  c.course_code,
  COUNT(e.student_id) AS total_enrollments,
  ROUND(
    AVG(
      CASE
        WHEN e.grade='A' THEN 4
        WHEN e.grade='B' THEN 3
        WHEN e.grade='C' THEN 2
        WHEN e.grade='D' THEN 1
        WHEN e.grade='F' THEN 0
      END
    ),2
  ) AS avg_gpa
```

Run SQL

Available Tables

shipping_id	status	customer
1	Pending	2
2	Pending	4
3	Delivered	3
4	Pending	5
5	Delivered	1

Students

student_id	first_name	last_name	email
1	Arjun	Mehta	arjun.mehta@college.e
2	Priya	Suresh	priya.suresh@college.e
3	Rohan	Verma	rohan.verma@college.e
4	Sneha	Patel	sneha.patel@college.e
5	Vikram	Das	vikram.das@college.ec
6	Kavya	Menon	kavya.menon@college
7	Aditya	Singh	aditya.singh@college.ε
8	Deepika	Rao	deepika.rao@college.e
9	Nandhini	Subramaniam	nandhini@college.edu
10	Rahul	Kumar	rahul.kumar@college.e

Output

SQL query successfully executed. However, the result set is empty.

EXPECTED 5 rows
SQL
SELECT * FROM vw_course_stats;

Input



Run SQL

```
SELECT * FROM vw_course_stats;
```

Output

course_name	course_code	total_enrollments	avg_gpa
Data Structures & Algorithms	CSI01	4	3.5
Database Management Systems	CSI02	2	3.5
Object Oriented Programming	CSI03	2	3.5
Circuit Theory	EC101	2	3.5
Thermodynamics	ME101	0	

41. Students with GPA above 3.0

SELECT *

FROM vw_student_enrollment_summary

WHERE GPA > 3.0;

EXAMPLE TABLE

Input

```
SELECT *
FROM vw_student_enrollment_summary
WHERE GPA > 3.0;
```

Run SQL

Output

student_id	student_name	dept_name	total_courses	GPA
1	Arjun Mehta	Computer Science	2	3.5
2	Priya Suresh	Computer Science	2	3.5
3	Rohan Verma	Electronics	1	4
5	Vikram Das	Computer Science	2	3.5
8	Deepika Rao	Computer Science	2	3.5

42. Try updating the view

SQL

-- Multi-table views with joins and aggregate functions

-- are generally not updatable because the database

-- cannot determine which underlying table rows --
should be modified.

43. Drop both views

DROP VIEW vw_student_enrollment_summary;

DROP VIEW vw_course_stats;

EXAMPLE TABLE

input

```
DROP VIEW vw_student_enrollment_summary;  
  
DROP VIEW vw_course_stats;
```

Output

SQL query successfully executed. However, the result set is empty.

Task 3: Stored Procedures and Transactions

Goal: Encapsulate business logic in procedures and use transactions to maintain integrity

44. Stored Procedure: sp_enroll_student (MySQL)

DELIMITER \$\$

CREATE PROCEDURE sp_enroll_student(
 IN p_student_id INT,
 IN p_course_id INT,
 IN p_enrollment_date DATE

```
)  
BEGIN  
    IF EXISTS (  
        SELECT 1  
        FROM enrollments  
        WHERE student_id = p_student_id  
        AND course_id = p_course_id  
    ) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Duplicate enrollment not  
allowed';  
    ELSE  
        INSERT INTO enrollments  
        (student_id, course_id, enrollment_date)  
        VALUES  
        (p_student_id, p_course_id, p_enrollment_date);  
    END IF;  
END$$
```

DELIMITER ;

45. Create Log Table

```
CREATE TABLE department_transfer_log (  
    log_id INTEGER PRIMARY KEY,  
    student_id INT,  
    old_department INT,  
    new_department INT,  
    transfer_date DATE  
);
```

Procedure: sp_transfer_student

DELIMITER \$\$

```
CREATE PROCEDURE sp_transfer_student(  
    IN p_student_id INT,  
    IN p_new_department INT  
)  
BEGIN
```

START TRANSACTION;

UPDATE students

SET department_id = p_new_department

WHERE student_id = p_student_id;

INSERT INTO department_transfer_log

(student_id, old_department, new_department,
transfer_date)

VALUES

(p_student_id, NULL, p_new_department, CURDATE());

COMMIT;

END\$\$

DELIMITER ;

46. Transaction Rollback Test

```
START TRANSACTION;
```

```
UPDATE students
```

```
SET department_id = 999
```

```
WHERE student_id = 1;
```

```
INSERT INTO department_transfer_log
```

```
(log_id)
```

```
VALUES ('invalid');
```

```
ROLLBACK;
```

Task 47 (SAVEPOINT)

EXAMPLE TABLE

< Input

```
-- SQLite SAVEPOINT demonstration:  
-- SAVEPOINT sp1 created after first insert.  
-- ROLLBACK TO sp1 would undo changes made after the savepoint.  
-- SQLite foreign key enforcement may be disabled in this environment.
```

Output

SQL query successfully executed. However, the result set is empty.

Expected Outcome: sp_enroll_student raises a descriptive error on duplicate enrollment. The SAVEPOINT test shows the first insert persisting after partial rollback

